

Rosemary Beetle Thesis Code

```

1  Sys.setenv(LANG = "en")
2  setwd("D:/IMPERIAL/.70062_Research_Project/Datasets")
3
4  library(dplyr)
5  library(stringr)
6  library(sf)
7  library(ggplot2)
8  library(glmTMB)
9  library(ggeffects)
10 library(sjPlot)
11
12 # =====
13 # PART 1: GEOGRAPHIC FRAMEWORK & SPATIAL CHECKPOINT (CACHED)
14 # =====
15 # This section stitches together England, Wales, Scotland, and Northern Ireland
16 # into one unified UK map. Because shapefiles are massive, we use an if/else
17 # statement to only build this once. If the .rds file exists, it skips the math.
18 # =====
19 cat("\n--- CHECKING FOR SPATIAL MASTER ---\n")
20 spatial_rds <- "Boundaries/UK_Spatial_Master.rds"
21
22 if (file.exists(spatial_rds)) {
23   uk_spatial_master <- readRDS(spatial_rds)
24   cat("Loaded existing UK_Spatial_Master.rds (Skipping geometry build)\n")
25 } else {
26   cat("Building Spatial Master from scratch... this may take a minute.\n")
27
28   # A. Load and format population data for England & Wales
29   ew_pop <- read.csv("Boundaries/Wards/Ward_pop_estimate.csv", sep=";", check.names = FALSE) %>%
30     dplyr::mutate(Total_Pop = as.numeric(stringr::str_replace_all(Total, " ", ""))) %>%
31     dplyr::select(WD23CD = `Ward 2023 Code`, Total_Pop)
32
33   ew_boundaries <- read.csv("Boundaries/Wards/WD_MAY_2023_UK_BGC_OnlyEngland&Wales.csv", sep=";",
34     check.names = FALSE)
35
36   # Calculate baseline metrics: Area and Population Density per Ward
37   ew_master <- ew_boundaries %>%
38     dplyr::left_join(ew_pop, by = "WD23CD") %>%
39     dplyr::filter(!is.na(Total_Pop)) %>%
40     dplyr::mutate(
41       ID = dplyr::row_number(),
42       Area_SqKm = Shape__Area / 1000000,
43       Pop_Density_SqKm = Total_Pop / Area_SqKm
44     ) %>%
45     dplyr::select(ID, Zone_Code = WD23CD, Zone_Name = WD23NM, Total_Pop, Area_SqKm,
46       Pop_Density_SqKm)
47
48   # B. Load Scotland and Northern Ireland equivalent census areas
49   scot_data <-
50     read.csv("Boundaries/SG_IntermediateZoneBdry_2022/SG_IntermediateZoneBdry_2022_MHW_ExportTable.csv",
51       sep = ";", check.names = FALSE) %>%
52     dplyr::select(Zone_Code = IZCode, Zone_Name = IZName, Total_Pop = TotPop2022, Area_SqKm =
53       StdAreaKM2, Pop_Density_SqKm = `Population_density_km2`)
54
55   ni_data <- read.csv("Boundaries/geography-sdz2021-esri-shapefile/SDZ2021_ExportTable.csv", sep =
56     ";", check.names = FALSE) %>%
57     dplyr::select(Zone_Code = SDZ2021_cd, Zone_Name = SDZ2021_nm, Total_Pop = All_usual_residents,
58       Area_SqKm = Area_SqKm, Pop_Density_SqKm = `Population_density_km2`)

```

```

52
53 uk_master <- dplyr::bind_rows(ew_master, scot_data, ni_data) %>%
54   dplyr::mutate(ID = dplyr::row_number()) %>%
55   dplyr::select(ID, Zone_Code, Zone_Name, Total_Pop, Area_SqKm, Pop_Density_SqKm)
56
57 # C. Load the physical polygon geometries (Shapefiles)
58 ew_sf <- sf::st_read("Boundaries/Wards/WD_MAY_2023_UK_BGC.shp", quiet = TRUE) %>%
59   sf::st_transform(27700) %>% # 27700 is the British National Grid (meters)
60   dplyr::select(Zone_Code = WD23CD) %>%
61   dplyr::filter(grepl("^[EW]", Zone_Code))
62
63   scot_sf <-
64   sf::st_read("Boundaries/SG_IntermediateZoneBdry_2022/SG_IntermediateZoneBdry_2022_MHW.shp", quiet =
65   TRUE) %>%
66   sf::st_transform(27700) %>%
67   dplyr::select(Zone_Code = IZCode)
68
69   ni_sf <- sf::st_read("Boundaries/geography-sdz2021-esri-shapefile/SDZ2021.shp", quiet = TRUE) %>%
70   sf::st_transform(27700) %>%
71   dplyr::select(Zone_Code = SDZ2021_cd)
72
73 # D. Bind all geometries and scale the population density for statistical modeling
74 uk_spatial_master <- rbind(ew_sf, scot_sf, ni_sf) %>%
75   dplyr::left_join(uk_master, by = "Zone_Code") %>%
76   dplyr::mutate(Pop_Density_Z = scale(Pop_Density_SqKm)[,1]) %>%
77   dplyr::filter(!is.na(Total_Pop))
78
79 # Save the checkpoint so we never have to run this block again
80 saveRDS(uk_spatial_master, spatial_rds)
81 cat("Saved new UK_Spatial_Master.rds\n")
82 }
83
84 # =====
85 # PART 2: BIOLOGICAL & CLIMATE DATA EXTRACTION
86 # =====
87 # Here we map the literal X/Y coordinates of the beetle sightings and the Met
88 # Office climate grids into our census polygons using spatial intersections.
89 # =====
90 cat("\n--- PROCESSING BIOLOGICAL & CLIMATE DATA ---\n")
91
92 # Load points
93 temp_grid <- read.csv("ClimateData_MetOffice/Annual_Average_Temperature_Change.csv", sep=";")
94 grid_sf <- sf::st_as_sf(temp_grid, coords = c("square_lon", "square_lat"), crs = 4326) %>%
95   sf::st_transform(27700)
96
97 ca_pts <- read.csv("Chrysolina/Chrysolina.csv", sep=";") %>%
98   sf::st_as_sf(coords = c("Chryso_lon", "Chryso_lat"), crs = 4326) %>%
99   sf::st_transform(27700)
100
101 pc_pts <- read.csv("Pyrochroa/Pyrochroa.csv", sep=";") %>%
102   sf::st_as_sf(coords = c("Pyro_lon", "Pyro_lat"), crs = 4326) %>%
103   sf::st_transform(27700)
104
105 # Extract local temperatures: Assigns the nearest Met Office grid to each Ward
106 zone_centroids <- sf::st_centroid(uk_spatial_master)
107 zone_climate <- sf::st_join(zone_centroids, grid_sf, join = sf::st_nearest_feature) %>%
108   sf::st_drop_geometry() %>%
109   dplyr::select(Zone_Code, Temp_Hist = baseline_1981_2000) %>%
110   dplyr::filter(!is.na(Temp_Hist)) %>%
111   dplyr::mutate(Temp_Hist_Z = scale(Temp_Hist)[,1])

```

```

111 # Aggregate target species (C. americana) counts per Ward, per Year (1994-2021)
112 ca_zone <- sf::st_join(ca_pts, uk_spatial_master, join = sf::st_intersects) %>%
113   sf::st_drop_geometry() %>%
114   dplyr::mutate(Year = Chryso_year) %>%
115   dplyr::filter(Year >= 1994 & Year <= 2021 & !is.na(Zone_Code)) %>%
116   dplyr::group_by(Zone_Code, Year) %>%
117   dplyr::summarise(Count_Ca = dplyr::n(), .groups = "drop")
118
119 # Aggregate calibration species (P. coccinea) to measure observer bias
120 pc_zone <- sf::st_join(pc_pts, uk_spatial_master, join = sf::st_intersects) %>%
121   sf::st_drop_geometry() %>%
122   dplyr::mutate(Year = Pyro_year) %>%
123   dplyr::filter(Year >= 1994 & Year <= 2021 & !is.na(Zone_Code)) %>%
124   dplyr::group_by(Zone_Code, Year) %>%
125   dplyr::summarise(Count_Pc = dplyr::n(), .groups = "drop")
126
127
128 # =====
129 # PART 3: MASTER PANEL & GIS SUMMARIES (AND MEMORY CLEANUP)
130 # =====
131 # We must create a "zero-inflated" panel. If a ward wasn't visited in 1998, R
132 # doesn't know it exists. Expand_grid forces R to acknowledge the zeroes (absences).
133 # =====
134 cat("\n--- BUILDING MASTER PANELS & SUMMARIES ---\n")
135
136 active_zones <- unique(c(ca_zone$Zone_Code, pc_zone$Zone_Code))
137
138 # A. Longitudinal Panel for Statistical Models (Ward + Year)
139 master_panel <- tidyr::expand_grid(Zone_Code = active_zones, Year = 1994:2021) %>%
140   dplyr::left_join(ca_zone, by = c("Zone_Code", "Year")) %>%
141   dplyr::left_join(pc_zone, by = c("Zone_Code", "Year")) %>%
142   dplyr::left_join(zone_climate, by = "Zone_Code") %>%
143   dplyr::left_join(sf::st_drop_geometry(uk_spatial_master)[, c("Zone_Code", "Pop_Density_Z")], by =
144     "Zone_Code") %>%
145   dplyr::mutate(
146     # Clean NAs into true ecological zeroes
147     Count_Ca = ifelse(is.na(Count_Ca), 0, Count_Ca),
148     Count_Pc = ifelse(is.na(Count_Pc), 0, Count_Pc),
149     # Create Binary Presence/Absence for the Hurdle Model
150     Presence_Ca = ifelse(Count_Ca > 0, 1, 0),
151     Presence_Pc = ifelse(Count_Pc > 0, 1, 0),
152     # Center the year so the model doesn't mathematically calibrate to Year 0 AD
153     Year_Centered = Year - 1994
154   ) %>%
155   dplyr::filter(!is.na(Temp_Hist_Z) & !is.na(Pop_Density_Z))
156
157 # B. Zone Summaries for Final Map Export (Collapsed across all years)
158 ca_totals <- ca_zone %>%
159   dplyr::group_by(Zone_Code) %>%
160   dplyr::summarise(Total_Ca = sum(Count_Ca), .groups = "drop")
161
162 zone_gis_summary <- sf::st_drop_geometry(uk_spatial_master) %>%
163   dplyr::left_join(ca_totals, by = "Zone_Code") %>%
164   dplyr::mutate(
165     Total_Ca = ifelse(is.na(Total_Ca), 0, Total_Ca),
166     Density_Coefficient = Total_Ca / Pop_Density_SqKm # Observer bias metric
167   ) %>%
168   dplyr::select(Zone_Code, Total_Ca, Density_Coefficient)
169
170 # STREAMLINING: Free up RAM before running heavy statistics
171 # Removed zone_climate, ca_zone, and pc_zone from the kill list so Part 6 doesn't crash!

```

```

171 rm(ew_boundaries, ew_master, ew_pop, ew_sf, ni_data, ni_sf, scot_data, scot_sf,
172     ca_pts, pc_pts, temp_grid, grid_sf)
173 gc() # Garbage collection
174
175
176 # =====
177 # PART 4: STATISTICAL ANALYSIS (THERMAL / PRESENCE MODEL)
178 # =====
179 cat("\n--- CHECKING THERMAL MODEL ---\n")
180 thermal_rds <- "thermal_model.rds"
181
182 if (file.exists(thermal_rds)) {
183     thermal_model <- readRDS(thermal_rds)
184     cat("Loaded existing thermal_model.rds\n")
185 } else {
186     cat("Calculating thermal model...\n")
187     # A binomial GLM to determine the strict climate gateway for initial establishment
188     thermal_model <- glm(
189         Presence_Ca ~ Temp_Hist_Z + Presence_Pc,
190         family = binomial(link = "logit"),
191         data = master_panel
192     )
193     saveRDS(thermal_model, thermal_rds)
194 }
195 print(summary(thermal_model))
196
197
198 # =====
199 # PART 5: ABUNDANCE MODEL & INTERACTION PLOT
200 # =====
201 cat("\n--- CHECKING ABUNDANCE MODEL & PREDICTIONS ---\n")
202 abundance_rds <- "abundance_model_tmb.rds"
203 predictions_rds <- "abundance_predictions_tmb.rds"
204
205 if (file.exists(abundance_rds)) {
206     abundance_model_tmb <- readRDS(abundance_rds)
207     cat("Loaded existing abundance_model_tmb.rds\n")
208 } else {
209     cat("Calculating abundance model (this is heavy, please wait)...\n")
210     # Negative Binomial (nbinom2) handles overdispersed ecological counts.
211     # (1 | Zone_Code) is the random intercept controlling for spatial clustering.
212     abundance_model_tmb <- glmmTMB::glmmTMB(
213         Count_Ca ~ Year_Centered * Pop_Density_Z + Temp_Hist_Z + Count_Pc + (1 | Zone_Code),
214         data = master_panel, family = nbinom2
215     )
216     saveRDS(abundance_model_tmb, abundance_rds)
217 }
218 print(summary(abundance_model_tmb))
219
220 # =====
221 # H3: DENSITY-DEPENDENT ABUNDANCE PLOT (UPDATED)
222 # =====
223
224 # Cache the plotting math with an explicit condition for zero observer bias
225 if (file.exists(predictions_rds)) {
226     predictions_tmb <- readRDS(predictions_rds)
227 } else {
228     predictions_tmb <- ggeffects::ggpredict(
229         abundance_model_tmb,
230         terms = c("Year_Centered", "Pop_Density_Z [-1, 0, 2]"),
231         condition = c(Count_Pc = 0) # Forces the plot to assume zero observer bias

```

```

232   )
233   saveRDS(predictions_tmb, predictions_rds)
234 }
235
236 # 1. Create a label dataframe with fixed coordinates for the bottom right
237 label_data <- as.data.frame(predictions_tmb) %>%
238   dplyr::group_by(group) %>%
239   dplyr::filter(x == min(x) | x == max(x)) %>%
240   dplyr::arrange(x) %>%
241   dplyr::summarize(
242     slope = (log10(dplyr::last(predicted)) - log10(dplyr::first(predicted))) / (dplyr::last(x) -
dplyr::first(x)),
243     .groups = "drop"
244   ) %>%
245   dplyr::mutate(
246     group_letter = dplyr::case_when(
247       group == "-1" ~ "(a)",
248       group == "0" ~ "(b)",
249       group == "2" ~ "(c)"
250     ),
251     label_text = sprintf("%s Slope: %.3f", group_letter, slope),
252     # Set fixed X and Y coordinates to place the text in the bottom right corner
253     # Y coordinates are spaced logarithmically to appear evenly distributed
254     text_x = 2020,
255     text_y = dplyr::case_when(
256       group == "-1" ~ 0.018, # Top label
257       group == "0" ~ 0.006, # Middle label
258       group == "2" ~ 0.002 # Bottom label
259     )
260   )
261
262 # 2. Build the updated plot
263 abundance_plot <- ggplot2::ggplot(predictions_tmb, ggplot2::aes(x = x + 1994, y = predicted, color =
group)) +
264   ggplot2::geom_line(linewidth = 1.2) +
265   ggplot2::geom_ribbon(ggplot2::aes(ymin = conf.low, ymax = conf.high, fill = group), alpha = 0.1,
color = NA) +
266
267   # Add the generated text labels inside the plot space
268   ggplot2::geom_text(
269     data = label_data,
270     ggplot2::aes(x = text_x, y = text_y, label = label_text),
271     hjust = 1, # Right-aligns the text against the text_x coordinate
272     show.legend = FALSE,
273     fontface = "bold",
274     size = 4
275   ) +
276
277   # Updated legend labels with the specific SD and National Average notation
278   ggplot2::scale_color_manual(
279     values = c("#3182bd", "#756bb1", "#de2d26"),
280     labels = c("(a) Rural (-1 SD)", "(b) Suburban (0 SD, National Average)", "(c) Urban (+2 SD)")
281   ) +
282   ggplot2::scale_fill_manual(
283     values = c("#3182bd", "#756bb1", "#de2d26"),
284     guide = "none"
285   ) +
286
287   # Restored standard X-axis limits since the text is now safely inside the plot
288   ggplot2::scale_x_continuous(
289     breaks = seq(1995, 2020, by = 5),

```

```

290     limits = c(1994, 2023)
291   ) +
292
293   ggplot2::scale_y_log10() +
294
295   # Updated Y-axis title
296   ggplot2::labs(
297     title = "Density-Dependent Expansion of C. americana (1994-2021)",
298     subtitle = "Predicted abundance index highlighting rates of population increase across
urbanisation levels",
299     x = "Year",
300     y = "Predicted Annual Abundance Index (Log10)",
301     color = "Census Area Classification:"
302   ) +
303
304   ggplot2::theme_minimal(base_size = 14) +
305   ggplot2::theme(
306     legend.position = "bottom"
307   )
308
309   print(abundance_plot)
310
311
312   # =====
313   # PART 6: FUTURE CLIMATE PROJECTIONS
314   # =====
315   cat("\n--- GENERATING FUTURE CLIMATE PREDICTIONS ---\n")
316
317   # Re-extract the original climate means to ensure standardized future math
318   temp_hist_vector <- sf::st_drop_geometry(uk_spatial_master) %>%
319     dplyr::left_join(zone_climate, by = "Zone_Code") %>% dplyr::pull(Temp_Hist)
320
321   temp_mean <- mean(temp_hist_vector, na.rm = TRUE)
322   temp_sd <- sd(temp_hist_vector, na.rm = TRUE)
323
324   # Base template for forecasting (Holds Year at 27 (2021) and Observer Bias at 0)
325   base_pred_data <- uk_spatial_master %>%
326     sf::st_drop_geometry() %>%
327     dplyr::left_join(zone_climate, by = "Zone_Code") %>%
328     dplyr::mutate(Year_Centered = 27, Count_Pc = 0)
329
330   future_predictions_table <- base_pred_data %>% dplyr::select(Zone_Code)
331
332   # Predict Baseline
333   future_predictions_table$Pred_Abund_Baseline <- predict(abundance_model_tmb, newdata =
base_pred_data, type = "response", re.form = NA)
334
335   # STREAMLINING: Loop through scenarios instead of repeating code blocks
336   scenarios <- c(1.5, 2.5, 4.0)
337   for (temp_inc in scenarios) {
338     # sprintf("%.1f") forces R to keep the trailing .0 for the 4.0 scenario
339     formatted_temp <- sprintf("%.1f", temp_inc)
340     col_name <- paste0("Pred_Abund_", gsub("\\.", "", formatted_temp), "C")
341
342     # Standardize the artificially warmed temperature
343     temp_scenario_data <- base_pred_data %>%
344       dplyr::mutate(Temp_Hist_Z = ((Temp_Hist + temp_inc) - temp_mean) / temp_sd)
345
346     # Predict and attach to table
347     future_predictions_table[[col_name]] <- predict(abundance_model_tmb, newdata = temp_scenario_data,
type = "response", re.form = NA)

```

```

348 }
349
350
351 # =====
352 # PART 7: MASTER UNIFIED SPATIAL EXPORT
353 # =====
354 cat("\n--- CHECKING GIS EXPORT ---\n")
355 map_rds <- "Boundaries/Future_Projections_Data.rds"
356
357 if (file.exists(map_rds)) {
358   final_master_map <- readRDS(map_rds)
359   cat("Loaded existing Future_Projections_Data.rds. (Delete file to regenerate GeoPackage)\n")
360 } else {
361   cat("Building and exporting final GeoPackage to ArcGIS...\n")
362
363   # Combine geometries, historical summaries, and future predictions
364   final_master_map <- uk_spatial_master %>%
365     dplyr::left_join(zone_gis_summary, by = "Zone_Code") %>%
366     dplyr::left_join(future_predictions_table, by = "Zone_Code") %>%
367     dplyr::select(Zone_Code, Zone_Name, Total_Pop, Pop_Density_SqKm, Pop_Density_Z,
368                 Total_Ca, Density_Coefficient, Pred_Abund_Baseline,
369                 Pred_Abund_15C, Pred_Abund_25C, Pred_Abund_40C)
370
371   # Dummy scale locks ArcGIS symbology color ramps across multiple layers
372   final_master_map$Dummy_Scale <- seq(0, 20, length.out = nrow(final_master_map))
373
374   # Export map to the Outputs directory
375   sf::st_write(final_master_map, "../Outputs/Chrysolina_Invasion_Map.gpkg", layer = "main", append =
FALSE)
376   saveRDS(final_master_map, map_rds)
377   cat("Success! Combined spatial layer written to GeoPackage layer 'main' in the Outputs folder.\n")
378 }
379
380 # =====
381 # PART 8: EXPORTING PUBLICATION-READY TABLES
382 # =====
383 cat("\n--- EXPORTING TABLES ---\n")
384
385 # Export to the Outputs directory
386 sjPlot::tab_model(thermal_model,
387                   digits = 3,
388                   file = "../Outputs/Table_1_Thermal_Model.html",
389                   title = "Table 1: Logistic Hurdle Model (Presence Probability)")
390
391 sjPlot::tab_model(abundance_model_tmb,
392                   digits = 3,
393                   file = "../Outputs/Table_2_Abundance_Model.html",
394                   title = "Table 2: Negative Binomial Count Model (Abundance Index)")
395
396 cat("\n--- PIPELINE COMPLETE! ---\n")

```